



Python Web & Getting Started with FastAPI

Web Design and Programming

M.Sc. Le Duy Khanh

Faculty of Data Science and Artificial Intelligence



Session Outline

1. From Frontend to Backend

2. Virtual Environments & Packages

3. WSGI, ASGI, and Uvicorn

4. Your First FastAPI Application

5. `def` vs `async def`

7. Project Structure

8. Strategy for DS & AI Students

9. Worked Example & Labs

10. Review & Summary



Learning Objectives

By the end of this session, you will be able to:

1. Describe what a web server program does: receive a request, route it, return a response with a status code.
2. Create a Python virtual environment, install packages, and record them in `requirements.txt`.
3. Explain WSGI vs. ASGI and the role of Uvicorn.
4. Write and run a minimal FastAPI app with one endpoint.
5. Decide between `def` and `async def` for a path function.
6. Open `/docs` and call your own API from it.
7. Split the group project into a frontend and a backend part.



How to Approach This Session

- This is the **pivot session**: from now on the backend is Python + FastAPI.
- You already know the language – FastAPI just *attaches* your Python functions to URLs.
- `async def` + `await` is the **same idea** as Week 5's `async` function – only the syntax changes.
- Today's end goal: a GET / endpoint running locally, and /docs open in your browser.



From Frontend to Backend

The Server's Job

- Weeks 1-5: the **browser (client)** was the centre. Now we write the program on the other end of `fetch()`.
- The request/response cycle, from the server side:
 1. Client sends an HTTP request: verb + path, query params, headers, optional body.
 2. Server **routes** it – matches verb + path to one function.
 3. The function does the work and **returns a value**.
 4. Framework packages it into a response: body (usually JSON), headers, status code.
- A framework's core job: **translate between HTTP bytes and Python data structures**.



Why This Matters for a DS/AI Project

- Your ML model is a Python function: inputs $\rightarrow$ prediction.
- A web service is just a way to expose that function over HTTP.
- FastAPI is the thinnest reasonable layer between “a Python function” and “an HTTP endpoint”.



Virtual Environments & Packages

Isolate Each Project with `venv`

```
1 python -m venv .venv          # create environment in ./venv
2 source .venv/bin/activate     # activate (macOS / Linux)
3 # .venv\Scripts\activate      # activate (Windows PowerShell)
4 deactivate                    # leave the environment
```

- Once activated, pip install writes into .venv; python runs that interpreter.
- Avoids package-version conflicts between projects and the system Python.

Install and Record Dependencies

```
1 pip install fastapi "uvicorn[standard]"
2
3 pip freeze > requirements.txt          # packages + exact versions
4 pip install -r requirements.txt        # a teammate reproduces it
```

- fastapi – the framework; uvicorn[standard] – the ASGI server + --reload.
- Add .venv/ to .gitignore; commit requirements.txt, not the environment.
- Modern alternatives: uv, Poetry, Pipenv (course uses venv + pip).



WSGI, ASGI, and Uvicorn

WSGI vs. ASGI

	WSGI	ASGI
Model	Synchronous	Asynchronous
Used by	Flask, classic Django	FastAPI (via Starlette)
While waiting	Worker blocked (busy waiting)	Event loop serves other requests
<code>async def</code>	Not native	Native

- Web work is mostly **waiting** (DB, network) – ASGI is why FastAPI is one of the fastest Python frameworks.
- Same **event loop** idea as JavaScript in Week 5.



Uvicorn – the ASGI Server

```
1 uvicorn main:app --reload
```

- main – the module (main.py); app – the FastAPI() object inside it.
- --reload – restart on file change. **Development only** (see Week 13).
- Default address: `http://127.0.0.1:8000`.
- FastAPI ships **no** server of its own – Uvicorn runs the app.



Your First FastAPI Application

A Minimal main.py

```
1 from fastapi import FastAPI
2 app = FastAPI()
3
4 @app.get("/")
5 def read_root():
6     return {"message": "Hello , Web!"}
7
8 @app.get("/hello/{name}")          # path parameter
9 def hello(name: str):
10     return {"greeting": f"Hello {name}"}
11
12 @app.get("/add")                  # query params: /add?a=2&b=3
13 def add(a: int, b: int):
14     return {"a": a, "b": b, "sum": a + b}
```



What Each Piece Does

- `app = FastAPI()` – the top-level application object.
- `@app.get("/")` – a **path operation decorator**: bind GET / to the function below (also post/put/patch/delete). The function under it is the **path function**.
- `{name}` in the path → **path parameter**; params not in the path → **query parameters**.
- Type hints give free validation: `/add?a=x&b=3` → clear 422, not a crash.
- Return a dict/list/model → **auto-converted to JSON**; default status 200.

def vs async def

Which One to Write?

```
1 import asyncio
2 from fastapi import FastAPI
3 app = FastAPI()
4
5 @app.get("/slow")
6 async def slow():
7     await asyncio.sleep(1)    # stand-in for a real slow I/O call
8     return {"done": True}
```

- `async def` when the body needs `await`; otherwise `def`.
- FastAPI runs `async def` on the event loop, plain `def` in a **threadpool** – neither blocks. You never call `asyncio.run()` yourself.
- **Week 5 link:** declare `async`, `await` the slow call, caller awaits you – identical thinking.



Automatic Documentation



A First Look at /docs

With the server running, open:

- <http://127.0.0.1:8000/docs> – **Swagger UI**: every endpoint, its parameters, and a “Try it out” button that sends real requests.
- <http://127.0.0.1:8000/redoc> – **ReDoc**: a read-oriented view.
- <http://127.0.0.1:8000/openapi.json> – the raw **OpenAPI** description.
- Generated from your function signatures + type hints – nothing extra to write.
- Deeper dive (tags, descriptions, testing): Week 10.



Project Structure

Minimal Backend Layout + Repo Split

```
1 project/  
2 |-- frontend/    # HTML/CSS/JS from Weeks 2-5  
3 |-- backend/  
4     |-- .venv/      # not committed (.gitignore)  
5     |-- main.py     # FastAPI app  
6     |-- requirements.txt # pinned dependencies
```

- Two folders in one repo, or two repos – agree as a team.
- Assign a backend owner.
- The frontend keeps calling APIs with `fetch()`; from Week 7 those are the APIs *you* write.



Strategy for DS & AI Students



Learning Strategy

- **You already know the language** – functions, dict/list, type hints. No new language this week.
- **Compare with Flask:** both bind routes with a decorator. FastAPI adds type validation (Pydantic), auto-docs, native async. Flask reads a global request; FastAPI declares typed parameters.
- `async def` **is a familiar face** – put it next to Week 5's async function.
- **Aim to run something today:** a GET / endpoint + /docs open.



Worked Example & Labs

Worked Example: A Two-Endpoint Service

```
1 from fastapi import FastAPI
2 app = FastAPI()
3 __notes = [{ "id": 1, "text": "First note" },
4             { "id": 2, "text": "Second note" }]
5
6 @app.get("/notes")
7 def list_notes():
8     return __notes
9
10 @app.get("/notes/{note_id}")
11 def get_note(note_id: int):
12     for n in __notes:
13         if n["id"] == note_id:
14             return n
15     return { "error": "not found" }
```

- Test /notes, /notes/1, and /docs. (Proper 404 + Pydantic bodies: Week 7.)

Lab 1: Environment Setup

Hands-on practice

- Create and activate `.venv`.
- `pip install fastapi "uvicorn[standard]"`.
- `pip freeze > requirements.txt`.
- Create `.gitignore` containing `.venv/`.

Lab 2: First Endpoints

Hands-on practice

- GET / returning a greeting dict.
- GET /hello/{name} – a path parameter.
- GET /add?a=2&b=3 – query parameters, returns the sum.
- Run `uvicorn main:app --reload`; test in the browser *and* via /docs.
- Add one `async def` endpoint that `await asyncio.sleep(1)`.

Lab 3: Project Integration

Hands-on practice

- As a group, create `backend/`, add the FastAPI skeleton, make the first commit.
- Split the repository into `frontend/` and `backend/`.
- Assign the backend owner(s) for the coming weeks.



Review & Summary

Quick MCQ Review

1. What does `python -m venv` do? **Creates an isolated Python environment per project**
2. Which standard does FastAPI need for native `async def`? **ASGI**
3. Uvicorn's role? **The ASGI server that runs the app**
4. What does `@app.get("/items")` declare? **The function below handles GET /items**
5. Uvicorn's default port? **8000**
6. When to use `--reload`? **Development only**
7. Return a dict from a path function – client gets? **JSON, status 200**

Self-Check Checklist

- ☐ Can create/activate a `venv` and produce `requirements.txt`
- ☐ Can explain WSGI vs. ASGI and what Uvicorn does
- ☐ Can write and run a minimal FastAPI app with Uvicorn
- ☐ Know when to use `def` vs. `async def`
- ☐ Can open `/docs` and call an endpoint from it
- ☐ Group project is split into `frontend/` and `backend/`



Summary and Next Session

- This week **pivots** from client to server: receive a request, route it, return a response + status code.
- venv isolates packages; requirements.txt makes it reproducible; .venv/ is never committed.
- **WSGI** (sync) vs. **ASGI** (async); FastAPI is ASGI via Starlette; **Uvicorn** runs it (uvicorn main:app --reload).
- Minimal app = FastAPI() + **path operation decorators** on **path functions**; return value auto-converts to JSON.
- async def when the body needs await; else def – same thinking as Week 5, in Python.
- /docs gives an interactive Swagger UI for free.
- Next: **FastAPI Routing & Request/Response** – path/query params, Pydantic bodies, response_model, status codes (Week 7).

Questions & Answers